

Phase 1 Completion Report — Nucleus AI SaaS Transformation

Date: June 3, 2026

Phase: 1 (SaaS Foundation: Auth, RBAC, Admin Panel, Security Hardening)

Status:  Complete

1. Executive Summary

Phase 1 transformed Nucleus AI from a single-tenant prototype into a **multi-user SaaS platform** with a complete authentication system, role-based access control (RBAC), an admin panel, and production-grade security hardening.

What was accomplished

- **Authentication:** Email/password auth with email verification, password reset, change password, account logout, and session invalidation.
- **RBAC:** 4-tier role hierarchy (VIEWER → EDITOR → ADMIN → SUPER_ADMIN) with a granular permission matrix enforced on every API route.
- **Admin Panel:** Dashboard with platform stats, full user management (CRUD, suspend/activate, invite), and brand management.
- **Security Hardening:** Rate limiting on auth endpoints, Zod input validation across routes, XSS sanitization, and a full set of HTTP security headers (CSP, HSTS, X-Frame-Options, etc.).
- **Testing:** All critical flows verified end-to-end against a running build.

Production Readiness Score: 88 / 100

Category	Score	Notes
Authentication	95	Complete & tested; email delivery needs SMTP config in prod
Authorization (RBAC)	95	Enforced server-side on all routes
Input Validation	90	Zod on auth + brand routes; a few legacy routes still light
Security Headers	90	CSP is functional but uses <code>unsafe-inline</code> (Next.js requirement)
Rate Limiting	80	In-memory (single-instance); needs Redis for multi-instance
Testing	75	Manual E2E verified; automated test suite not yet added for frontend
Overall	88	Production-ready for single-instance deploy; see Next Steps

2. Implemented Features

Feature	Status	Notes
Email/password signup	✓ Working	Defaults to VIEWER, PENDING_VERIFICATION
Email verification	✓ Working	Token-based; activates account
Resend verification	✓ Working	Anti-enumeration generic response
Login (NextAuth Credentials)	✓ Working	JWT sessions, remember-me
Account lockout	✓ Working	5 failed attempts → 30-min lock
Forgot password	✓ Working	1-hour reset token, anti-enumeration
Reset password	✓ Working	Invalidates all sessions on reset
Change password	✓ Working	Requires current password
Session invalidation	✓ Working	<code>sessionVersion</code> bump = logout everywhere
RBAC permission matrix	✓ Working	4 roles, 22 granular permissions
Server-side route guards	✓ Working	<code>requirePermission</code> / <code>requireRole</code>
Client-side permission hooks	✓ Working	<code>usePermissions()</code> for UI gating
Admin dashboard (stats)	✓ Working	Users by role/status, brands, campaigns
Admin user management (CRUD)	✓ Working	Create/list/update/delete + filters
Admin user suspend/activate	✓ Working	Dedicated endpoints
Admin user invite	✓ Working	Invite via email token
Admin brand management	✓ Working	List/manage all brands
Privilege-escalation guard	✓ Working	ADMIN cannot assign/manage SUPER_ADMIN

Feature	Status	Notes
Audit logging	✅ Working	All sensitive actions recorded
Rate limiting (auth)	✅ Working	5/min login & signup; 3/hr reset
Input validation (Zod)	✅ Working	Auth + brand routes
XSS sanitization	✅ Working	HTML stripped from text inputs
Security headers	✅ Working	CSP, HSTS, X-Frame-Options, etc.
Email delivery (SMTP)	⚠️ Partial	Works when SMTP configured; dev mode logs token
Automated frontend tests	❌ Not Done	Manual E2E only (deferred to Phase 2)

3. Database Changes

Migration applied: 20260603062639_phase1_rbac_overhaul

New enums

- `Role` — VIEWER, EDITOR, ADMIN, SUPER_ADMIN
- `UserStatus` — ACTIVE, SUSPENDED, PENDING_VERIFICATION
- `CampaignStatus`, `DocumentType`, `ProcessingStatus`, `NotificationType`

New models

Model	Purpose
<code>AuditLog</code>	Records sensitive actions (who/what/when + IP)
<code>Notification</code>	In-app notifications
<code>Document</code>	Context Vault documents (vs. legacy <code>UploadedDocument</code>)

Fields added to `User`

- `role` (Role, default VIEWER), `status` (UserStatus, default ACTIVE)
- `emailVerified`, `emailVerificationToken`
- `passwordResetToken`, `passwordResetExpires`
- `lastLogin`, `loginAttempts`, `lockedUntil`

- `sessionVersion` (for logout-everywhere)
- Indexes on `role` and `status`

Fields added to existing models

- `BrandProfile` : `createdBy` (creator tracking)
- `Campaign` : `budget` , `start_date` , `end_date` (backend Attribution Engine)

4. Files Created / Modified

Created (this phase — Part 3)

File	Description
<code>frontend/src/lib/rate-limit.ts</code>	In-memory sliding-window rate limiter + IP helper + policies
<code>frontend/src/lib/sanitize.ts</code>	XSS sanitization helpers (<code>sanitizeText</code> , <code>sanitizeObject</code>)
<code>frontend/src/lib/validations/auth.ts</code>	Zod schemas for all auth payloads
<code>PHASE1_COMPLETION_REPORT.md</code>	This report

Modified (Part 3)

File	Change
frontend/next.config.ts	Added full security-headers config (CSP, HSTS, etc.)
frontend/src/middleware.ts	Added rate limiting for auth endpoints
frontend/src/app/api/signup/route.ts	Zod validation + XSS sanitization
frontend/src/app/api/auth/login/route.ts	Zod validation
frontend/src/app/api/auth/forgot-password/route.ts	Zod validation
frontend/src/app/api/auth/reset-password/route.ts	Zod validation
frontend/src/app/api/auth/change-password/route.ts	Zod validation
frontend/src/app/api/auth/resend-verification/route.ts	Zod validation
frontend/src/app/api/brands/route.ts	Zod validation + sanitization
frontend/src/app/api/brands/[id]/route.ts	Zod validation + sanitization

Created in earlier Parts 1 & 2 (reference)

- frontend/src/lib/permissions.ts — RBAC matrix
 - frontend/src/middleware/rbac.ts — server-side guards
 - frontend/src/hooks/usePermissions.ts — client permission hook
 - frontend/src/lib/audit.ts, email.ts, tokens.ts — supporting libs
 - frontend/src/app/admin/** — admin pages (dashboard, users, brands)
 - frontend/src/app/api/admin/** — admin APIs
 - frontend/src/app/api/auth/** — auth API routes
 - Auth pages: login, signup, verify-email, forgot-password, reset-password, settings/password
-

5. Security Improvements

#	Improvement	Detail
1	Rate limiting	Login & signup: 5 req/min; password reset & resend: 3 req/hr. Returns HTTP 429 with <code>Retry-After</code> .
2	Input validation	Zod schemas on all auth routes + brand routes. Strict email/password policy (≥8 chars, letter+number).
3	XSS sanitization	HTML tags, <code><script></code> , <code>javascript:</code> URIs, and inline event handlers stripped from text inputs.
4	Security headers	CSP, <code>Strict-Transport-Security</code> (HSTS), <code>X-Frame-Options: DENY</code> , <code>X-Content-Type-Options: nosniff</code> , <code>Referrer-Policy</code> , <code>Permissions-Policy</code> , <code>X-XSS-Protection</code> .
5	Account lockout	5 failed logins → 30-minute lock.
6	Password hashing	bcrypt with cost factor 12.
7	Session invalidation	<code>sessionVersion</code> enables “logout from all devices”; password reset bumps it.
8	Anti-enumeration	Forgot-password & resend-verification return generic messages.
9	Privilege-escalation guard	ADMIN cannot create/modify SUPER_ADMIN; role-management rules enforced server-side.
10	Audit logging	Signup, login, password changes, user/brand mutations recorded with IP.
11	RBAC enforcement	

#	Improvement	Detail
		Every API route guarded server-side; UI gated client-side (defense in depth).

6. Credentials for Testing

Seeded via `npx prisma db seed` (frontend). **Change these in production!**

Role	Email	Password
SUPER_ADMIN	<code>superadmin@nucleus-ai.com</code>	<code>SuperAdmin123!</code>
ADMIN	<code>admin@nucleus-ai.com</code>	<code>admin123</code>
EDITOR	<code>editor@nucleus-ai.com</code>	<code>editor1234</code>
VIEWER	<code>demo@nucleus-ai.com</code>	<code>demo1234</code>

7. Testing Results

All critical flows were verified end-to-end against a production build (`npm run build` + `npm start`):

Flow	Result
Signup → email verification → login	✓ Pass (user activated, session issued)
Input validation (bad email / weak password / bad JSON)	✓ Pass (400 with clear errors)
XSS sanitization (<code><script></code> in name)	✓ Pass (stripped)
Rate limiting (forgot-password)	✓ Pass (3 allowed, 4th → 429)
Security headers present	✓ Pass (all 7 headers)
Admin dashboard + pages load	✓ Pass (200 with admin session)
User CRUD (create/delete)	✓ Pass
Brand CRUD (create/delete)	✓ Pass
Privilege escalation (ADMIN → SUPER_ADMIN)	✓ Pass (403 blocked)
RBAC: VIEWER → admin APIs	✓ Pass (403)
RBAC: no session → protected APIs	✓ Pass (401)
Reset-password validation	✓ Pass
TypeScript compile (<code>tsc --noEmit</code>)	✓ Pass (0 errors)
Production build	✓ Pass

8. Known Issues

1. **Rate limiter is in-memory** — works for single-instance deploys, but resets on restart and isn't shared across instances. Use Redis (e.g. Upstash) for horizontal scaling. (Documented in `rate-limit.ts`.)
2. **CSP uses 'unsafe-inline'** — required by Next.js's inline runtime scripts/styles. A nonce-based CSP would be stricter (Phase 2 hardening).
3. **Email delivery requires SMTP config** — in dev (no SMTP), verification/reset tokens are returned in the API response for testing. Configure SMTP env vars in production.
4. **No automated frontend test suite** — flows were validated manually. Backend has pytest suites; frontend Jest/Playwright tests are deferred to Phase 2.
5. **Some legacy routes lightly validated** — `documents`, `upload/*`, and `workflow/execute` rely on RBAC + light checks; consider full Zod schemas in Phase 2.

9. Next Steps for Phase 2

1. **Multi-tenancy / Organizations** — introduce an `Organization` model and scope brands/campaigns/documents per org (team workspaces).
2. **Billing & subscriptions** — Stripe integration, plan tiers, usage metering.
3. **Distributed rate limiting** — swap in Redis/Upstash for the limiter.
4. **Automated test suite** — Jest unit tests + Playwright E2E for the frontend; wire into CI.
5. **Nonce-based CSP** — remove `'unsafe-inline'` for stricter XSS protection.
6. **OAuth providers** — Google/GitHub social login.
7. **2FA / MFA** — TOTP-based two-factor authentication.
8. **Notification delivery** — surface the `Notification` model in the UI + email digests.
9. **Full Zod coverage** — validation schemas on remaining API routes.
10. **Observability** — wire up Sentry + structured logging (configs already documented in `docs/deployment/monitoring.md`).

10. Environment Variables Required

See `ENV_SETUP.md` for full details. Phase 1 essentials:

Frontend (`frontend/.env.local`)

Variable	Required	Purpose
<code>DATABASE_URL</code>	✓	PostgreSQL (Prisma)
<code>NEXTAUTH_SECRET</code>	✓	Session JWT signing (<code>openssl rand -base64 32</code>)
<code>NEXTAUTH_URL</code>	✓	App canonical URL
<code>NEXT_PUBLIC_API_URL</code>	✓	Backend API base URL
<code>SMTP_HOST</code> / <code>SMTP_PORT</code> / <code>SMTP_USER</code> / <code>SMTP_PASSWORD</code> / <code>SMTP_FROM</code>	⚠️ Prod	Email delivery (verification/reset)
<code>AWS_*</code>	Optional	S3 file uploads

Backend (backend/.env)

Variable	Required	Purpose
DATABASE_URL / DATA-BASE_URL_SYNC	✓	PostgreSQL (async + sync)
OPENAI_API_KEY	✓	Embeddings + LLM
QDRANT_URL / QDRANT_COLLECTION	✓	Vector DB
CORS_ORIGINS	✓	Must include frontend URL

Report generated at the close of Phase 1, Part 3 (Security Hardening, Testing & Reporting).